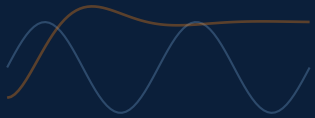
DAY 3 / 5

Mechatronics: Electrical Machines, Control, Servo Systems and Sensing



Advanced Control and Introduction to Servo Systems

Filters & compensation · state space · adaptive/robust · discrete-time control · servo architecture · inverter & DC bus

Mechatronics Training Programme

Theoretical Course Series ▪ Day 3 of 5

Agenda and learning objectives

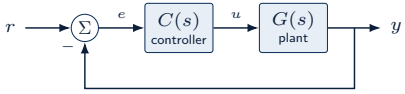
Code	Session	Time
2.6	Filters and compensation	60 min
2.7	Modern (state-space) control	60 min
2.8	Adaptive and robust control	60 min
2.9	Discrete-time control	60 min
3.1	Servo system architecture	30 min
3.2a	Rectifier, DC bus, inverter	30 min
	Net teaching time	**5 h**

Short breaks (≈10 min) after each 1-h block are extra.

By the end of today you can . . .

- **explain** what a filter does and **calculate** the phase it costs a loop
- **sketch** state feedback and **place** two poles by matching coefficients
- **explain** why gains must adapt or be robust when the load changes
- **calculate** the phase lost to sampling delay
- **draw** a servo drive from mains plug to motor shaft

What we learned so far (Day 2 in one slide)



- **feedback**: measure y, compare with r, correct — like cruise control
- **PID**: P reacts to error now, I to accumulated error, D to its trend
- loop gain $L(s) = C(s)G(s)$

- **Bode plot**: gain and phase of $L(j\omega)$ vs. frequency
- **crossover** ω_c: where $|L| = 1$ (0 dB) $\approx$ loop bandwidth
- **phase margin** PM $= 180^\circ + \angle L(j\omega_c)$: distance from instability
- rule: PM ≈ 45–60° for a well-damped response

Today's thread

Almost everything today either **costs phase** (filters, delay, resonances) or **buys it back** (lead, faster sampling).

Session 1

2.6 Filters and Compensation in Control Systems

Refresher: what does a filter do?

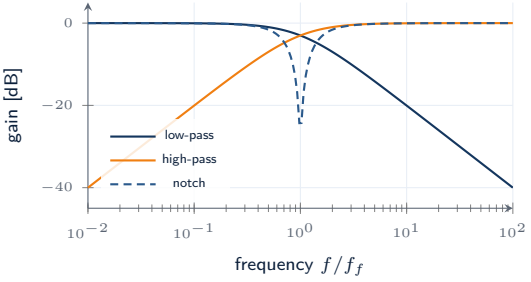
Refresher: a filter is a music equaliser

A **filter** passes some frequencies and weakens others.

- **low-pass**: treble down, bass kept
- **high-pass**: bass down (removes DC)
- **notch**: removes one tone (50 Hz hum)

$$|F(f)| = \frac{1}{\sqrt{1 + (f/f_f)^2}}, \qquad f_f = \frac{1}{2\pi RC}\ \text{[Hz]}$$

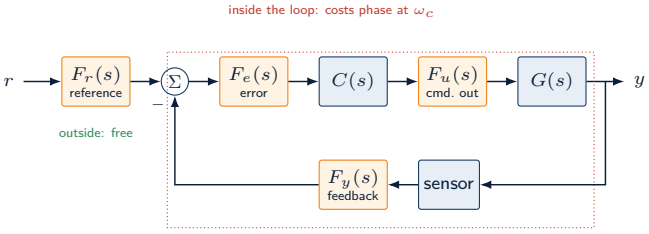
RC low-pass: $R = 1\,\text{k}\Omega$, $C = 1\,\mu\text{F} \Rightarrow f_f = 159\,\text{Hz}$; at 1.59 kHz only 0.1 passes.



Take-away

Every filter that weakens a signal also **delays** it a little. In a control loop that delay is a cost.

Why filters in a servo loop — and where they sit



What we filter against

- sensor noise (encoder steps, current sensor)
- mechanical resonances (belts, couplings)
- offsets of analogue sensors
- sudden steps in the command

Rule

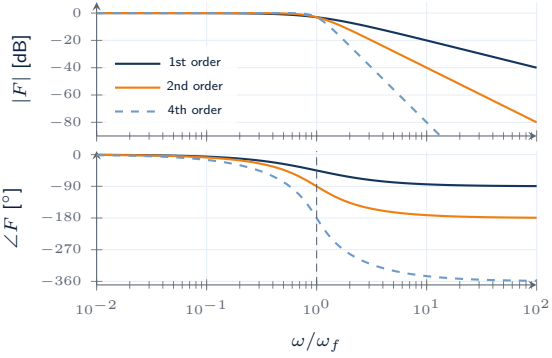
A filter **inside** the loop becomes part of $L(s)$ and eats phase margin. A filter on the reference (F_r) only shapes the response — no stability cost.

Low-pass filter: cut-off, order and phase

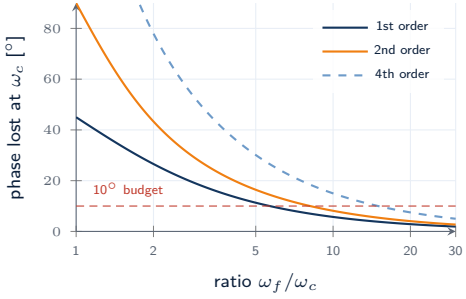
$$F_1(s) = \frac{\omega_f}{s + \omega_f}$$

- **cut-off** ω_f [rad/s]: the -3 dB point
- **order** n: steeper roll-off, $-20n$ dB per decade
- **phase**: $-45°$ per order at ω_f
- 2nd order with low damping **peaks**: it amplifies near ω_f

Take-away: higher order = cleaner signal but much more phase lag.



The price of filtering: phase lost at crossover



$$\Delta\varphi = \arctan\frac{\omega_c}{\omega_f} \quad \text{(1st-order LPF)}$$

Worked example. Velocity loop crossover $f_c = 150\,\text{Hz}$; we allow 10° loss.

1. need $f_c/f_f \le \tan 10^\circ = 0.176$
2. $f_f \ge 150\,\text{Hz}/0.176 = 850\,\text{Hz}$
3. check: $\arctan(150/850) = 10.0^\circ$ ✓

Trade-off

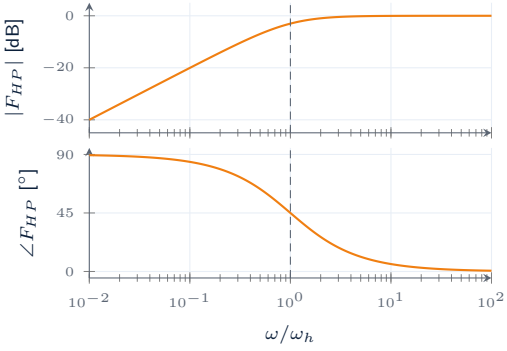
Lower ω_f = quieter motor but less phase margin. Rule of thumb: $\omega_f \approx (5\ldots10)\,\omega_c$.

High-pass filter: removing offsets

$$F_{HP}(s) = \frac{s}{s + \omega_h}$$

- blocks DC: removes sensor **offsets** and slow drift
- **washout**: passes only *changes* (vibration, acceleration)
- gives phase **lead** (up to $+90°$) below ω_h

Pitfall: never in the *main* feedback path — the loop would no longer see a constant position error and could not hold a load.

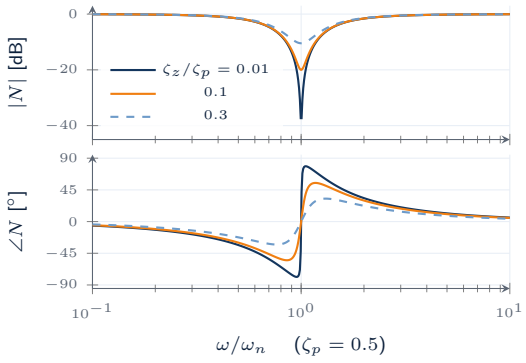


Notch filter: cutting out one frequency

Picture: a playground swing has one natural frequency; a mechanical axis too. A **notch** removes the controller's "push" at exactly that frequency.

$$N(s) = \frac{s^2 + 2\zeta_z \omega_n s + \omega_n^2}{s^2 + 2\zeta_p \omega_n s + \omega_n^2}$$

- **centre** ω_n: set to the resonance
- **depth**: ζ_z/ζ_p (e.g. $0.1 \to -20\,\text{dB}$)
- **width**: set by ζ_p — wide = robust to drift, but more phase lag

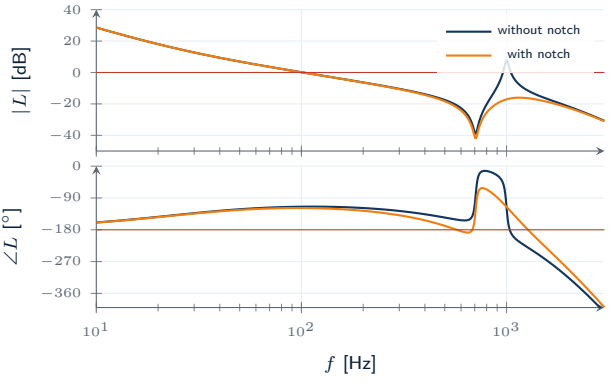


Notch in practice: taming a 1 kHz resonance

Axis: motor – flexible shaft – load, resonance at $f_r = 1\,\text{kHz}$. Velocity PI with $f_c = 100\,\text{Hz}$.

	no notch	notch
0 dB crossings	3	1
PM at 100 Hz	65°	61°
closed loop	unstable	stable

The resonance lifts $|L|$ above 0 dB near 1 kHz with no phase margin left. A $-25\,\text{dB}$ notch at 1 kHz removes that peak and costs only $\approx 4^\circ$ at 100 Hz.



Model and numbers computed for this course; cf. [Ellis, 2012, ch. 16].

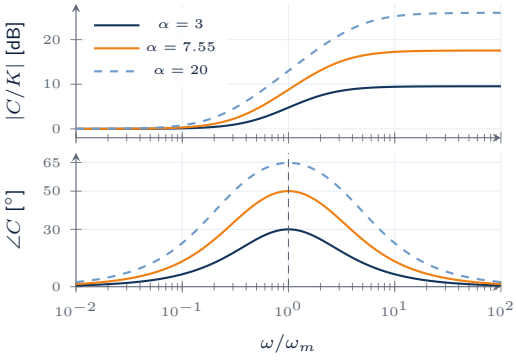
Phase-lead compensator: buying phase back

$$C_{\text{lead}}(s) = K\,\frac{1+\alpha T s}{1+Ts}, \qquad \alpha > 1$$

- a zero before a pole: phase goes **up** between them
- maximum lead at $\omega_m = 1/(T\sqrt{\alpha})$ — put it at ω_c
- $\alpha = 3 \to 30^\circ$, $\alpha = 7.5 \to 50^\circ$, $\alpha = 20 \to 65^\circ$
- price: high-frequency gain $\times\alpha \Rightarrow$ more noise

Take-away: a lead is a PD controller with a filtered D.

[Ogata, 2010, ch. 7]; [Franklin, Powell & Emami-Naeini, 2019, ch. 6].

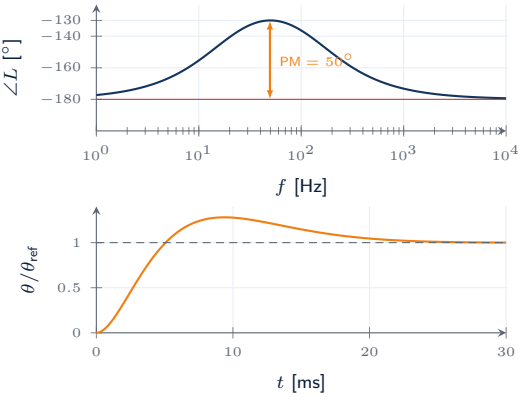


Lead design — worked example: position loop on a pure inertia

Plant $G = 1/(Js^2)$, $J = 2 \times 10^{-4}\,\text{kg m}^2$: phase is $-180°$ everywhere, so PM $= 0$ with a plain gain.
Goal: $f_c = 50\,\text{Hz}$ ($\omega_c = 314\,\text{rad/s}$), PM $= 50°$.

1. need $50°$ of lead $\Rightarrow \alpha = 7.55$ (table/graph)
2. centre it: $T = 1/(\omega_c\sqrt{\alpha}) = 1.16\,\text{ms}$
3. gain for $|L| = 1$ at ω_c: $K = J\omega_c^2/\sqrt{\alpha} = 7.2\,\text{N m/rad}$
4. check (computed): $f_c = 49.5\,\text{Hz}$, PM $= 50°$ ✓

Step overshoot 28 % (the lead zero adds some); a reference filter F_r can remove it.

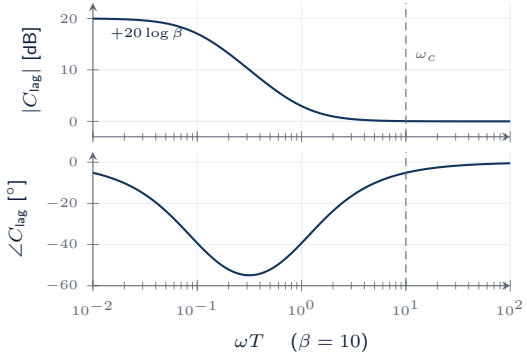


Phase-lag compensator and lead–lag

$$C_{\text{lag}}(s) = \beta\,\frac{1+Ts}{1+\beta Ts}, \qquad \beta > 1$$

- boosts gain at **low** frequency by $\beta \Rightarrow$ smaller steady error, stiffer against load
- leaves high frequencies (and noise) alone
- put its zero a decade below ω_c: costs only $\approx 5^\circ$
- **lead–lag**: lead buys PM, lag buys stiffness

PID is a lead–lag in disguise: PI $\approx$ lag, filtered D $\approx$ lead.



Session 2.6 summary and key terms

Key take-aways

- In-loop filters are part of $L(s)$: budget their phase ($\omega_f \approx 5$–$10\,\omega_c$).
- A notch removes one resonance for a few degrees of PM.
- Lead adds phase at ω_c; lag adds low-frequency stiffness.

Key terms

cut-off -3 dB frequency · **order** steepness of roll-off · **low/high-pass** keeps slow/fast parts · **notch** removes one frequency · **resonance** frequency where a structure swings strongly · **lead/lag** compensator adding phase / low-frequency gain

Check your understanding — filters and compensation

Check your understanding

1. A colleague adds a 4th-order low-pass at $2\,f_c$ to the velocity feedback "to clean it up". What happens to the phase margin?
2. Why does a reference filter F_r not change stability, but a feedback filter F_y does?
3. Your machine whistles at 800 Hz when you raise the gain. Which filter do you reach for, and where do you set its centre?

Discuss in pairs for 2 minutes, then we compare answers.

Session 2

2.7 Introduction to Modern Control

Refresher: matrices and eigenvalues

Refresher: eigenvalues

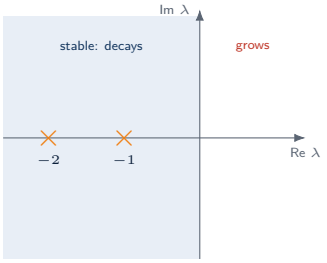
A matrix A maps a vector to a new vector. Special directions are only *stretched*:

$$A\boldsymbol{v} = \lambda \boldsymbol{v}$$

$\lambda =$ **eigenvalue**. Find them from $\det(\lambda I - A) = 0$.

Example: $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$

1. $\det(\lambda I - A) = \lambda(\lambda + 3) + 2 = \lambda^2 + 3\lambda + 2$
2. roots: $\lambda_1 = -1$, $\lambda_2 = -2$ [1/s]
3. so $\dot{\boldsymbol{x}} = A\boldsymbol{x}$ contains e^{-t} and e^{-2t}: both decay



Take-away

Eigenvalues of A = poles of the system. Like a guitar string's natural notes, they set how the system "rings" and whether it dies out (left half-plane).

State-space model of a servo axis

State $\boldsymbol{x}$: the smallest set of numbers that remembers everything about the system (here: where it is, how fast it moves).

$$\dot{\boldsymbol{x}} = A\boldsymbol{x} + Bu, \qquad y = C\boldsymbol{x}$$

Axis $J\dot{\omega} = -B_v\omega + u$ with $\boldsymbol{x} = [\theta \;\; \omega]^\mathsf{T}$, u = torque:

$$A = \begin{bmatrix} 0 & 1 \\ 0 & -a \end{bmatrix},\; B = \begin{bmatrix} 0 \\ b \end{bmatrix},\; C = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

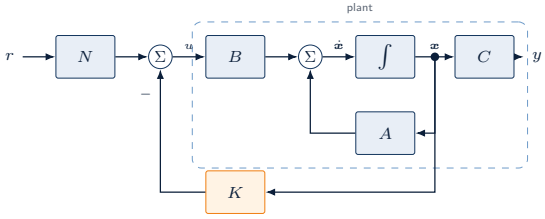
$a = B_v/J$, $b = 1/J$.

Numbers for this session:
$J = 1 \times 10^{-3}\,\mathrm{kg\,m^2}$, $B_v = 2 \times 10^{-3}\,\mathrm{N\,m\,s/rad}$
$\Rightarrow a = 2\,\mathrm{s}^{-1}$, $b = 1000\,1/(\mathrm{kg\,m^2})$.

- row 1: $\dot{\theta} = \omega$ (definition of speed)
- row 2: Newton for rotation
- one state per energy store (mass, inertia, inductor, …)
- same model works for many inputs and outputs

> Adding the motor current i as a third state gives a 3×3 model — same idea, one more row.

State feedback and pole placement



Picture: a driver who watches *both* the position and the speed of the car, not just the distance error.

$$u = -K\boldsymbol{x} + Nr$$
$$\Rightarrow\ \dot{\boldsymbol{x}} = (A - BK)\boldsymbol{x} + BN\,r$$

- new system matrix $A - BK$: its eigenvalues are the **closed-loop poles**
- choose $K \Rightarrow$ choose the poles (speed and damping)
- N scales r so that $y \to r$
- needs every state — measured or **estimated**

Limit: faster poles need more torque and amplify noise.

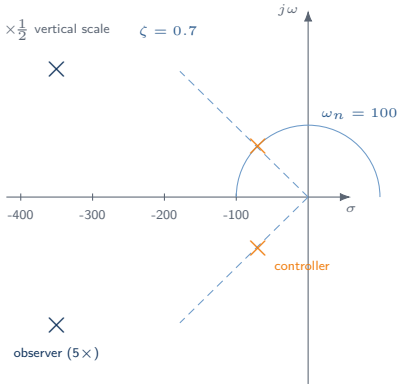
Worked example: placing the poles of the servo axis

Goal: $\omega_n = 100\,\text{rad/s}$, $\zeta = 0.7$. With $K = [k_1 \;\; k_2]$:

1. desired polynomial:
   $s^2 + 2\zeta\omega_n s + \omega_n^2 = s^2 + 140s + 10^4$
2. closed loop:
   $\det(sI - A + BK) = s^2 + (a + bk_2)s + bk_1$
3. match the last terms:
   $1000\,k_1 = 10^4 \Rightarrow k_1 = 10\,\text{N m/rad}$
4. match the middle terms:
   $2 + 1000\,k_2 = 140 \Rightarrow k_2 = 0.138\,\text{N m s/rad}$

$$K = [\,10 \quad 0.138\,]$$

Take-away: k_1 acts as a spring, k_2 as a damper — for this plant state feedback **is** PD control.



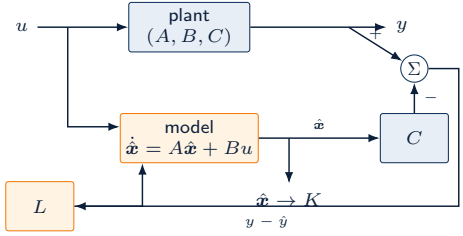
The observer: estimating states we do not measure

Picture: a ship's navigator without GPS uses a model (speed $\times$ time) and corrects it whenever a lighthouse is seen.

$$\dot{\hat{\boldsymbol{x}}} = \underbrace{A\hat{\boldsymbol{x}} + Bu}_{\text{model}} + \underbrace{L\,(y - C\hat{\boldsymbol{x}})}_{\text{correction}}$$

- estimation error dies out with poles $\operatorname{eig}(A - LC)$
- make them 2–6$\times$ faster than the controller poles
- **observable**: the measurement "sees" every state (encoder angle: yes; speed only: angle cannot be recovered)

[Luenberger, 1964]; [Franklin, Powell & Emami-Naeini, 2019, ch. 7].



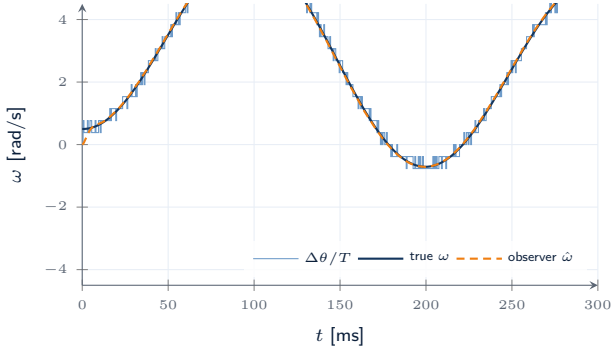
Separation principle: design K and L separately; together the poles are those of $A - BK$ plus those of $A - LC$.

Observer example: velocity from a 16-bit encoder

Same axis; only the angle θ is measured (65 536 counts/rev), sampled at 4 kHz.

1. **finite difference** $\Delta\theta/T$: one count $= 0.38\,\text{rad/s} \Rightarrow$ staircase noise
2. **observer** with poles at 500 rad/s (5× controller)
3. result: smooth $\hat{\omega}$; start-up error gone in $\approx$10 ms

Take-away: the observer uses the torque command too, so it lags less than "differentiate and filter".



Simulation for this course; observer-based velocity estimation: [Ellis, 2012, ch. 18].

Integral action in state feedback

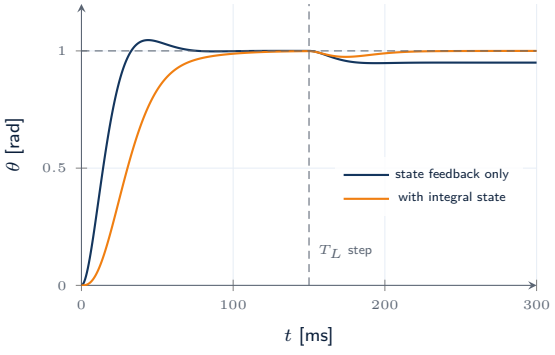
Add one extra state: the accumulated error.

$$\dot{z} = r - y, \qquad u = -k_1\theta - k_2\omega + k_I z$$

Third pole at -50 added to the earlier pair gives $k_1 = 17$, $k_2 = 0.188$, $k_I = 500$ (SI units).

Test: 1 rad step, then a load torque of 0.5 N m at 150 ms.

- without z ($k_I = 10$): error $T_L/k_1 = 0.05$ rad stays
- with z: error $\to 0$, just like the I in PI



Going further: controllability, Ackermann's formula and LQR

Going further — conceptual only: one picture, one idea. Not needed for the rest of the course.

Controllability and Ackermann

Controllable: the input can push every state anywhere. Test: $\mathcal{C} = [B \quad AB \quad \cdots]$ has full rank.

Ackermann's formula computes K directly from $\mathcal{C}$ and the desired poles — what software does for larger systems. For our 2×2 axis it gives the same $K = [10 \quad 0.138]$.

LQR: optimal state feedback

Instead of choosing poles, choose what you care about:

$$J = \int_0^\infty \left(\boldsymbol{x}^\mathsf{T} Q \boldsymbol{x} + u^\mathsf{T} R u \right) dt$$

large Q = "errors are expensive", large R = "torque is expensive". Software returns the best K; good margins come for free. Used for multi-mass axes and crane anti-sway.

[Ogata, 2010, ch. 10]; [Åström & Murray, 2021, ch. 7]; [Anderson & Moore, 1990].

Session 2.7 summary and key terms

Key take-aways

- $\dot{\boldsymbol{x}} = A\boldsymbol{x} + Bu$: one state per energy store.
- Poles = eigenvalues; $u = -K\boldsymbol{x}$ moves them to $\mathrm{eig}(A - BK)$.
- 2nd-order case: match coefficients to get K.
- An observer estimates unmeasured states (model + correction).
- An integral state removes steady error under load.

Key terms

state system memory · **eigenvalue** natural rate / pole · **state feedback** $u = -K\boldsymbol{x}$ · **pole placement** choosing K for given poles · **observer** model-based estimator · **separation principle** K and L designed independently

Check your understanding — state space

Check your understanding

1. A system has eigenvalues -5 and $+1\,\mathrm{s}^{-1}$. Is it stable? Which one causes trouble?
2. In the worked example, which gain do you increase to make the axis respond faster, and which to add damping?
3. Why do we place the observer poles faster than the controller poles — and why not $100\times$ faster?

Discuss in pairs for 2 minutes, then we compare answers.

Session 3

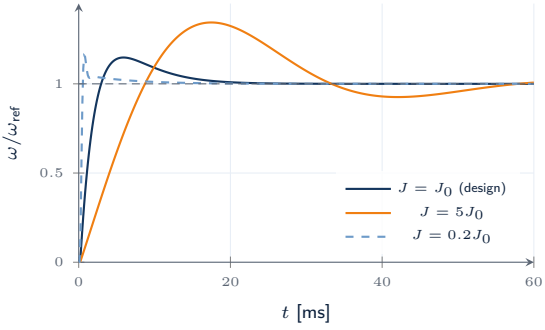
2.8 Introduction to Adaptive and Robust Control

Why fixed gains are not enough: parameter variation

Picture: a crane driver handles an empty hook and a 10-tonne load differently. A controller with fixed gains cannot.

Parameter	varies with
inertia J	payload, robot pose (1:10)
friction	temperature, wear
torque const. k_t	magnet temperature
resonance f_r	position, belt tension

Velocity PI tuned for J_0: with $5J_0$ the overshoot rises to **34 %**.



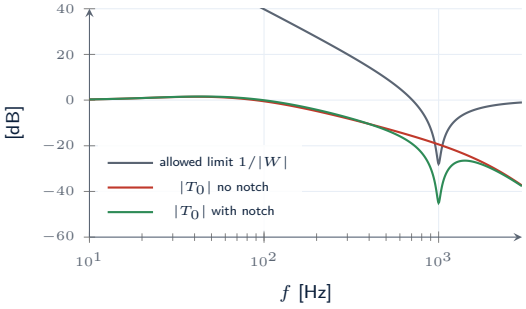
Two answers: **adapt** the controller to the actual parameters, or design **one robust** controller that works for all of them.

Robust control: one controller for a whole family of plants

Picture: a car suspension designed for an empty car *and* a full one — not perfect for either, safe for both.

- model uncertainty: the true G lies somewhere near our model G_0
- uncertainty **grows with frequency** (resonances, delays we did not model)
- so robustness is decided around and above ω_c
- price: robust designs are more cautious (lower bandwidth)

Plot: the 1 kHz resonance from Session 2.6. The closed-loop gain must stay below the grey "uncertainty" line — it does only with the notch.



Going further: this is the robust-stability test $|WT_0| < 1$ (small-gain theorem). H_∞ design finds the controller that best satisfies such tests automatically [Zhou, Doyle & Glover, 1996].

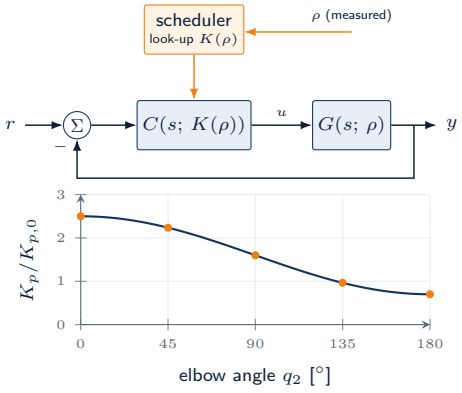
Gain scheduling: a pre-computed gain table

Picture: the gear you choose depends on the road slope — decided by a rule, not learned.

- tune the controller at a few operating points
- measure a **scheduling variable** ρ (robot angle, payload, roll diameter)
- interpolate the gains in between

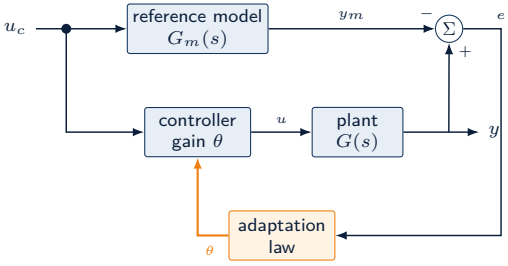
$$K_p(\rho) = \hat{J}(\rho)\,\omega_c \quad \text{(keeps } \omega_c \text{ constant)}$$

- simple, fast, easy to certify
- needs a model of $J(\rho)$; ρ must change slowly



illustrative 2-link arm; dots = tuned grid points

Model-reference adaptive control (MRAC)

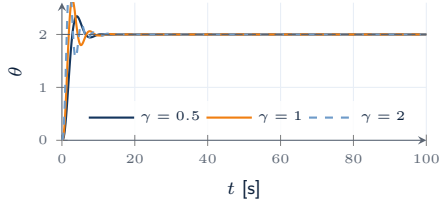


Picture: a learner driver copies the instructor (model); every mismatch e nudges their technique.

Classical example after [Åström & Wittenmark, 1995]; simulated for this course.

MIT rule: change the gain θ so that e^2 shrinks:

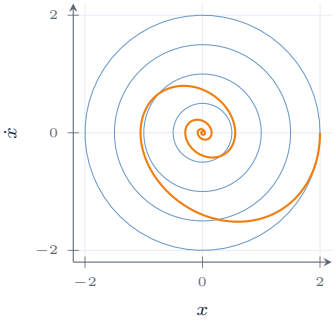
$$\frac{d\theta}{dt} = -\gamma\, e\, y_m$$



γ = learning rate: larger = faster, but too large can go unstable.

Going further: Lyapunov stability and provably stable adaptation

Going further — conceptual only: one picture, one idea. Not needed for the rest of the course.



damped oscillator: energy circles (blue), trajectory (orange) spirals inwards

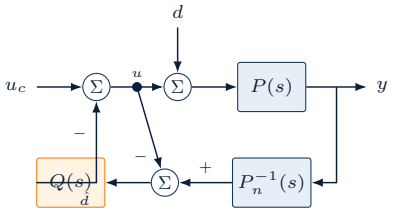
Idea: a marble in a bowl. If some "energy" V is always positive and *always decreasing*, the system must settle — we never need to solve the ODE.

$$V(\boldsymbol{x}) > 0, \qquad \dot{V}(\boldsymbol{x}) < 0 \;\Rightarrow\; \text{stable}$$

- **Lyapunov-based MRAC**: choose the adaptation law so that $\dot{V} \leq 0$ is guaranteed
- result looks like the MIT rule, but is proven stable for any learning rate $\gamma > 0$

[Slotine & Li, 1991, ch. 3 & 8]; [Åström & Wittenmark, 1995]; [Ioannou & Sun, 1996].

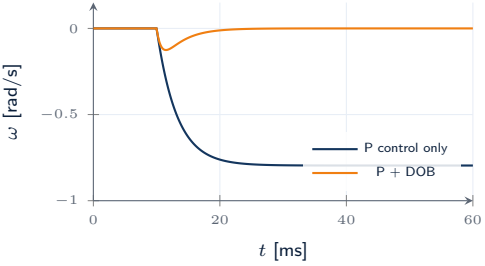
Disturbance observer (DOB)



Idea: "What torque *should* have produced this motion?" minus "what torque did I command?" $=$ the unknown disturbance $\hat{d}$. Subtract it.

$$\hat{d} = Q(s)\left[P_n^{-1}(s)\, y - u\right]$$

[Ohnishi, Shibata & Murakami, 1996].



Load step 0.5 N m at 10 ms: without DOB a speed error of -0.8 rad/s remains; with DOB it returns to 0.

- Q = low-pass; its bandwidth is limited by noise and delay

Session 2.8 summary and key terms

Approach	Idea	Trade-off
Robust	one fixed C that works for the whole family	safe, but less bandwidth
Gain scheduling	pre-computed gains vs. measured ρ	needs a model of $J(\rho)$
MRAC	adjust gains online so y follows y_m	learning transients, tuning γ
Disturbance observer	estimate the disturbance and cancel it	Q bandwidth vs. noise

Key take-aways

- Real parameters drift; fixed gains can lose performance.
- In drives: auto-tuning + scheduling + DOB are common; full MRAC is rare.

Key terms

uncertainty gap model–reality · **robust** works for all members · **adaptive** changes itself online · **reference model** desired behaviour

Check your understanding — adaptive and robust control

Check your understanding

1. A robot picks up a payload that triples the joint inertia. What happens to the crossover frequency of a fixed velocity PI, and why?
2. Gain scheduling vs. MRAC: which one "learns"? Which one would you rather certify for a safety-critical machine?
3. Why is model uncertainty usually largest at high frequency?

Discuss in pairs for 2 minutes, then we compare answers.

Session 4

2.9 Discrete-Time Control

Refresher: sampling a signal

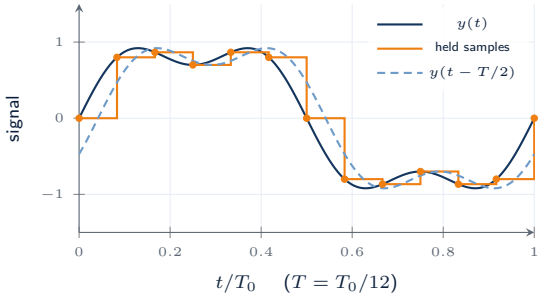
Refresher: sampling

A film camera shows motion as 24 photos per second. A digital controller sees a signal only at the instants $t = kT$:

$$f_s = \frac{1}{T} \quad [\text{Hz}], \qquad y_k = y(kT)$$

Between samples the output is **held** constant (zero-order hold, ZOH): a staircase.

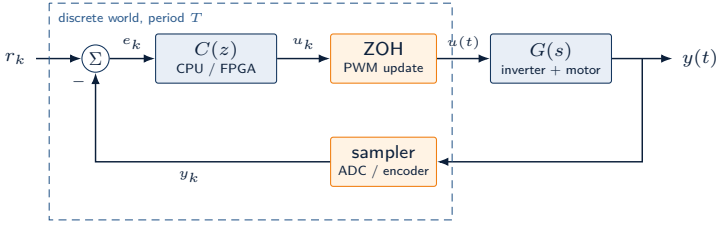
Example: $f_s = 8\,\text{kHz} \Rightarrow T = 125\,\mu\text{s}$; a 100 Hz sine gets 80 samples per period.



Take-away

The staircase follows the signal **half a sample late** on average: holding costs a delay of $T/2$.

Anatomy of a digital servo loop



- the computer sees y only at $t = kT$
- the output is held until the next update
- computing takes time: u_k is usually applied one sample later

Loop	typical f_s	typical f_c
current	8–32 kHz	1–3 kHz
velocity	4–16 kHz	100–500 Hz
position	1–8 kHz	20–100 Hz

Delay budget and phase loss at crossover

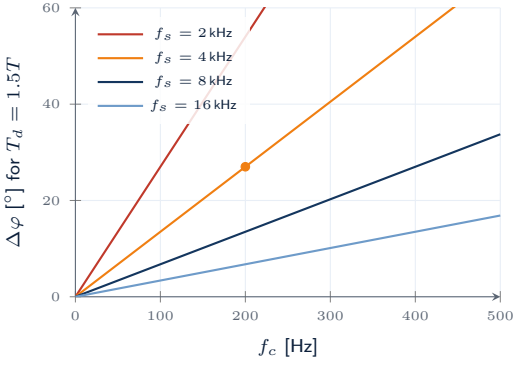
Picture: a shower with a long pipe — you turn the tap, the water changes later, and you overcorrect.

$$\Delta\varphi = 360^\circ \cdot f_c \cdot T_d, \qquad T_d \approx 1.5\,T$$

($T/2$ from the hold + T for computing.)

Worked example: $f_s = 4\,\text{kHz}$, $f_c = 200\,\text{Hz}$.

1. $T = 1/4000 = 250\,\mu\text{s}$
2. $T_d = 1.5 \times 250 = 375\,\mu\text{s}$
3. $\Delta\varphi = 360 \cdot 200 \cdot 0.000375 = \mathbf{27^\circ}$
4. at 8 kHz: 13.5° — double f_s, halve the loss



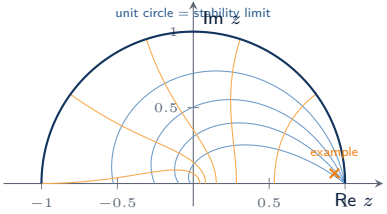
From the s-plane to the z-plane

In discrete time a pole is a number z that multiplies the signal every sample: $x_{k+1} = z\,x_k$.

$$z = e^{sT} \qquad \text{stable} \iff |z| < 1$$

- $|z| < 1$: each sample smaller $\Rightarrow$ decays
- left half s-plane $\rightarrow$ inside the unit circle
- near $z = 1$: slow; near $z = -1$: rings at $f_s/2$

Example: $s = -70 \pm j71$, $T = 1\,\text{ms}$
$\Rightarrow z \approx 0.93 \pm j0.07$, inside the circle: stable.



thin lines: constant damping (blue-grey) and constant frequency (amber)

Going further: discretisation methods and pre-warping

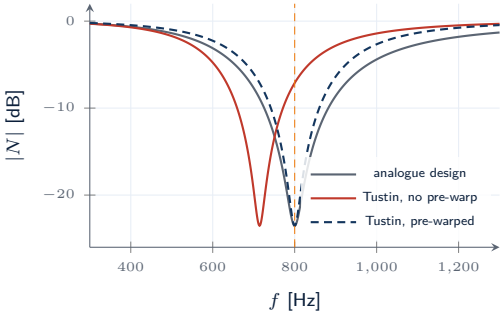
Going further — conceptual only: one picture, one idea. Not needed for the rest of the course.

To run $C(s)$ on a computer, replace s by an expression in z. Common choices:

- **forward Euler** — simplest, can turn a stable filter **unstable**
- **backward Euler** — always stable, slightly distorted
- **Tustin** (bilinear) — standard for PI/PID and filters
- **ZOH equivalent** — exact for plant models

Tustin squeezes frequencies near $f_s/2$: an 800 Hz notch at $f_s = 4$ kHz lands at 714 Hz unless it is **pre-warped**.

[Franklin, Powell & Workman, 1998, ch. 6]; [Oppenheim & Schafer, 2010, ch. 7].



Discrete PID implementation

Every T seconds the controller runs a few lines:

- **I**: add up the error: $I_k = I_{k-1} + K_i T\, e_k$
- **D**: difference of the measurement, filtered
- **saturation**: the output is limited (max. current)
- **anti-windup**: when saturated, stop the integrator growing — otherwise big overshoot later

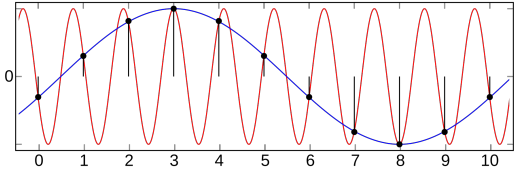
Picture: windup = keeping your foot down on the accelerator while the car is already at full power.

[Åström & Hägglund, 2006, ch. 3]; [Franklin, Powell & Workman, 1998].

```
// executed every T (ISR synced to PWM)
e  = r - y;
I += Ki*T*e;                        // integrate
D  = a*D - b*(y - y_old);           // filtered D on y
y_old = y;                 // a=Tf/(Tf+T), b=Kd/(Tf+T)
u  = Kp*e + I + D;
if (u > UMAX)      { I -= u - UMAX; u = UMAX; }
else if (u < -UMAX){ I -= u + UMAX; u = -UMAX;}
write_pwm(u);       // takes effect next period
```

D acts on $-y$ so a step in r does not cause a "kick".

Aliasing: the wagon-wheel effect



Moxfyre, CC BY-SA 3.0, via Wikimedia Commons

red: $f = 0.9$, blue: $f = 0.1$, sampled at $f_s = 1$ — identical samples

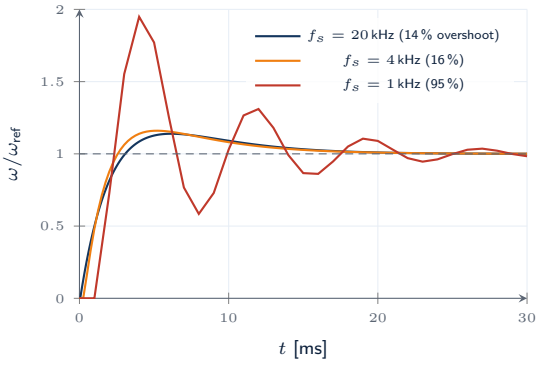
Picture: in films a fast wheel seems to turn slowly or backwards — the camera samples too slowly.

$$f_{\text{alias}} = |f - k\,f_s|\,, \quad k = \text{round}(f/f_s)$$

- anything above $f_s/2$ folds back below it
- 4.6 kHz vibration sampled at 4 kHz looks like **600 Hz**
- cure: filter *before* sampling (anti-alias RC filter)

More on Day 5 (sensing).

Choosing the sample rate



[Franklin, Powell & Workman, 1998, ch. 11]; simulation for this course.

Same velocity PI ($f_c = 100\,\text{Hz}$), only f_s changes.

At 1 kHz: $T_d = 1.5$ ms
$\Rightarrow \Delta\varphi = 360 \cdot 100 \cdot 0.0015 = 54^\circ \Rightarrow$ almost unstable.

Rules of thumb

- $f_s \geq 20$–$40\times$ the loop bandwidth
- check $360^\circ f_c T_d$ against the PM budget
- inner loops sample faster

Session 2.9 summary and key terms

Key take-aways

- Sampling + hold + computing $\approx 1.5T$ delay.
- Phase lost: $360^\circ f_c T_d$ — double f_s, halve it.
- Discrete stability: all poles inside the unit circle.
- Discrete PID needs saturation and anti-windup.
- Filter before sampling to avoid aliasing.

Key terms

sample rate $f_s = 1/T$ · **ZOH** holds the output between samples · z-**plane** map of discrete poles · **anti-windup** stops the integrator in saturation · **aliasing** fast signal masquerading as slow

Check your understanding — discrete-time control

Check your understanding

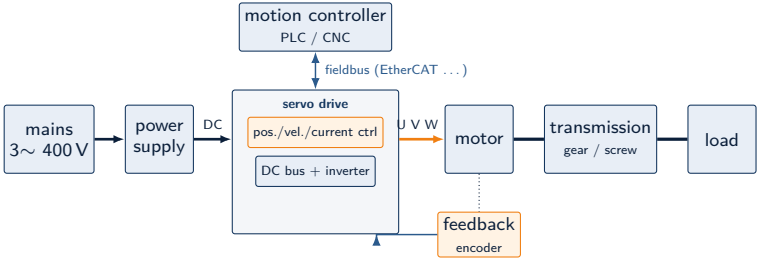
1. A current loop crosses at 2 kHz and is sampled at 16 kHz. How much phase does the $1.5T$ delay cost?
2. A discrete pole sits at $z = 1.05$. What does the signal do from one sample to the next?
3. Why does the integrator "wind up" when the drive hits its current limit?

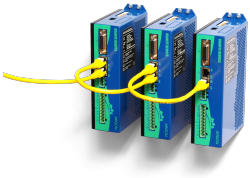
Discuss in pairs for 2 minutes, then we compare answers.

Session 5

3.1 Servo System Architecture

Servo system architecture: from mains to load





Tero Kontkanen, CC BY-SA 3.0, via Wikimedia Commons

- **drive** = power electronics + control loops + protection
- **motion controller** sends targets every 0.25–4 ms
- **feedback** (encoder) closes every loop



Projuktiponno, CC BY-SA 4.0, via Wikimedia Commons

compact servo drives on a fieldbus; incremental encoder

Torque, velocity and position control modes

Mode	Controller sends	Loops in the drive	Typ. BW	Example
Torque	torque T^*	current only	1–3 kHz	winder tension, press force
Velocity	speed ω^*	velocity + current	100–500 Hz	spindle, conveyor
Position	angle θ^*	position + velocity + current	20–100 Hz	robot, pick-and-place

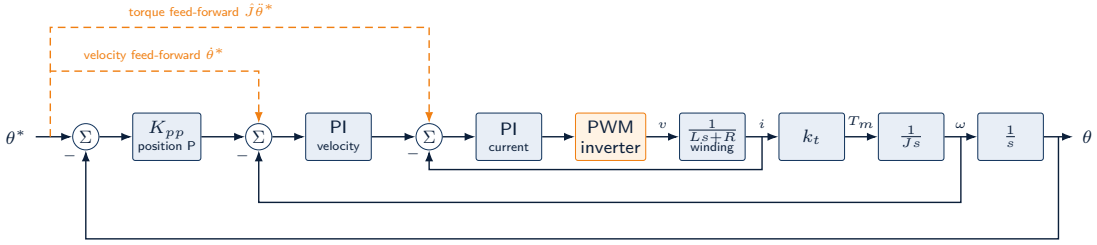
Analogy: telling a driver "press the pedal this much" (torque), "drive at 50 km/h" (velocity) or "stop at that line" (position).

Going further: fieldbus drive profiles (CiA 402) name these modes CST, CSV, CSP — Day 5.

Selection rule

Close each loop where its **sensor** is and where it can be updated **fast enough**.

Cascade control structure of a servo drive

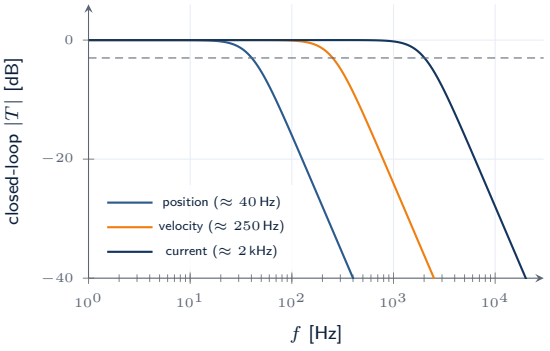


Picture: Russian dolls — each inner loop turns its part into something simple for the next one (current loop → "ideal torque source").

- each inner loop **3–10×** faster than the next
- tune from inside out: current → velocity → position
- feed-forward: act on the plan before an error appears

[Ellis, 2012, ch. 17]; [Leonhard, 2001].

Session 3.1 summary: the bandwidth hierarchy



Key take-aways

- Servo = supply + drive + motor + transmission + load + encoder + controller.
- Mode = which loops the drive closes.
- Cascade: current $\gg$ velocity $\gg$ position bandwidth.

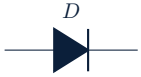
Key terms

cascade loops inside loops · **feed-forward** act on the reference · **bandwidth** how fast a loop follows

Session 6

3.2 Power Electronics I: Rectifier, DC Bus and Inverter

Refresher: diode, capacitor and transistor switch







Diode

A one-way **valve**: current flows only from anode to cathode. Turns AC into DC (rectifier).

Capacitor

An energy **tank**: smooths the voltage and buffers bursts of current.

$$E = \tfrac{1}{2} C V^2$$

Transistor switch

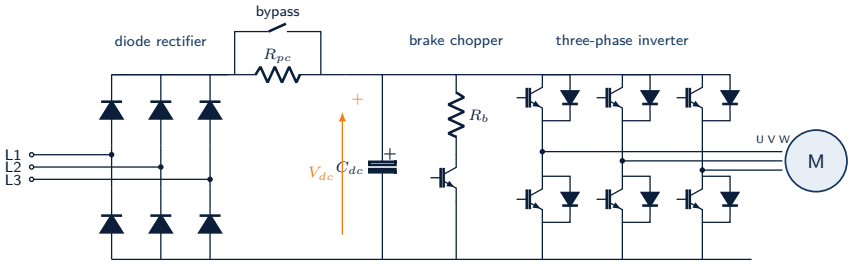
A tap opened by a small **gate** signal. Used only fully **on** or fully **off**.

Why only on/off? Power loss $P = V \cdot I$ in the switch. At $I = 10\,\text{A}$: on: $V \approx 1.5\,\text{V} \Rightarrow 15\,\text{W}$; off: $I \approx 0 \Rightarrow 0\,\text{W}$; half-way ($V = 280\,\text{V}$): **2.8 kW** — it would melt.

Take-away

A drive builds any voltage by switching very fast between on and off and averaging (PWM).

Power stage of a servo drive: overview

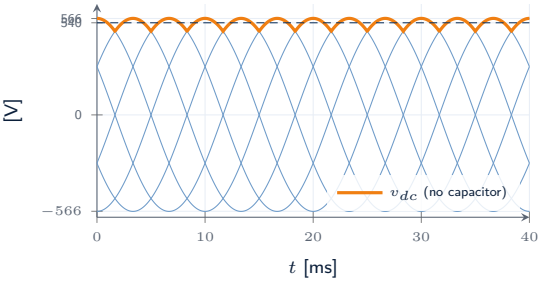




Projuktiponno, CC BY-SA 4.0, via Wikimedia Commons

AC $\rightarrow$ **DC** $\rightarrow$ **AC:** diodes rectify the mains $\rightarrow$ the capacitor stores it as a steady DC voltage $\rightarrow$ six transistor switches chop it into motor voltages of any amplitude and frequency. The brake chopper burns surplus energy when the motor brakes.

Rectifier and DC link



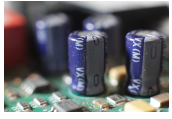
The diodes always pick the highest of the six line-to-line voltages: a bumpy DC.

$$V_{dc} \approx 1.35\, V_{LL} \text{ (average)}, \quad \hat{V} = \sqrt{2}\, V_{LL}$$

400 V mains: $1.35 \times 400 = 540$ V; capacitor charges to the peak ≈ 566 V.

- C_{dc} smooths the bumps
- pre-charge resistor limits the inrush into an empty C_{dc}
- diodes cannot send energy **back** to the mains





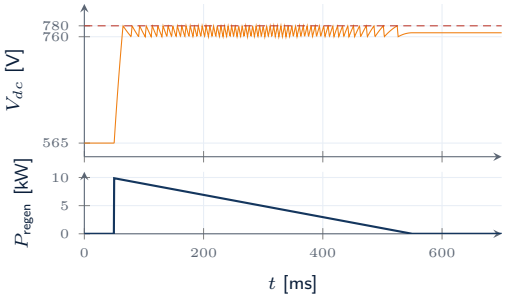
left: Hiro36, CC BY-SA 3.0; right: Giovanna 27, CC BY 4.0; via Wikimedia Commons

Regeneration and the brake chopper

Picture: braking a motor turns it into a generator — energy flows back into the drive.

Worked example: $J = 0.05\,\text{kg m}^2$ stops from 314 rad/s (3000 rpm) in 0.5 s.

1. energy: $E = \frac{1}{2}J\omega^2 = \frac{1}{2} \cdot 0.05 \cdot 314^2 = 2.47\,\text{kJ}$
2. capacitor (1 mF, 565→780 V) takes only $\frac{1}{2}C(780^2 - 565^2) = 145\,\text{J}$
3. peak power $P = J\alpha\,\omega = 31.4 \cdot 314 = 9.9\,\text{kW}$
4. resistor: $R_b \leq V^2/P = 780^2/9870 = 62\,\Omega \Rightarrow$ **56 Ω**

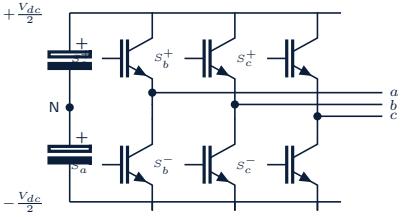


Chopper on at 780 V, off at 760 V.



Glogger, CC BY-SA 3.0, via Wikimedia Commons

Three-phase two-level inverter



freewheeling diodes omitted for clarity

- each **leg** = two switches; the output is tied to $+$ or $-$
- top and bottom are **never on together** (short circuit!)
- average output set by the on-time fraction (**duty cycle** d):

$$\bar{v}_{aN} = (2d - 1)\,\frac{V_{dc}}{2}$$

$d = 0.75$, $V_{dc} = 560\,\text{V}$: $\bar{v}_{aN} = 0.5 \cdot 280 = 140\,\text{V}$.



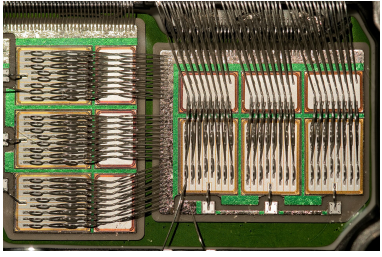
Sthdhk, CC BY-SA 4.0, via Wikimedia Commons

Power switches: IGBT, MOSFET and SiC

	Si IGBT	Si MOSFET	SiC MOSFET
voltage	600–1700 V	$\leq$ 250 V	650–1700 V
switching	slower	very fast	very fast
typ. f_{sw}	4–16 kHz	20–100 kHz	16–100 kHz
used in	400 V drives	24–72 V robots, AGVs	compact drives

- faster switching $\Rightarrow$ smoother current, faster current loop
- but steep voltage edges stress motor insulation and cause EMI

[Mohan, Undeland & Robbins, 2003, ch. 8 & 22]; [Krishnan, 2001].



Pslawinski, CC BY-SA 3.0, via Wikimedia Commons

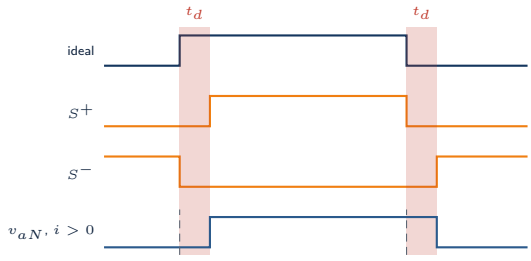
opened IGBT module: chips and bond wires

Gate drivers and dead time (preview)

Gate driver: the "muscle" between the microcontroller and the power switch.

- **isolation**: the top switch floats at hundreds of volts
- fast gate current (several A)
- short-circuit protection

Dead time t_d: a short pause with *both* switches off at every change, to avoid **shoot-through**.



Price: a small voltage error,

$$\Delta\bar{v} \approx \frac{t_d}{T_{sw}} V_{dc}$$

$t_d = 2\,\mu\text{s}$, $f_{sw} = 10\,\text{kHz}$ ($T_{sw} = 100\,\mu\text{s}$), $V_{dc} = 560\,\text{V} \Rightarrow$ **11.2 V** — compensated on Day 4.

[Balogh, TI SLUA618]; [Mohan, Undeland & Robbins, 2003].

Session 3.2 summary, key terms and check

Key take-aways

- Rectifier → DC link → brake chopper → inverter → motor.
- $V_{dc} \approx 1.35 V_{LL} = 540\,\text{V}$ at 400 V mains.
- Braking energy goes into R_b, not the capacitor.
- Switches only on/off; dead time prevents shoot-through.

Key terms

DC link capacitor bus · **leg** pair of switches · **duty cycle** on-time fraction · **shoot-through** both switches on

Check your understanding

1. Why can a servo drive not simply use a resistor-like "half-on" transistor to set the motor voltage?
2. A drive trips on DC-bus over-voltage only while braking. What is missing or too small?
3. In a cascade, which loop must be tuned first, and why?

Day 3 summary and glossary

Control

- filters cost phase; notches remove resonances; lead buys phase back
- state feedback places poles; observers estimate states
- robust vs. adaptive: one safe design vs. self-adjusting gains
- sampling delay $\approx 1.5T$ costs $360^\circ f_c T_d$

Servo hardware

- drive + motor + transmission + encoder + controller
- cascaded current/velocity/position loops
- rectifier, DC link, brake chopper, inverter, dead time

One sentence

Servo bandwidth is limited by phase: filters, delay and resonances spend it; compensators and fast sampling buy it back.

Preview of Day 4

Day 4 — Servo drives in depth

- PWM: how the inverter switches build a sine wave
- dead-time effects and compensation
- field-oriented control of a PMSM: i_d/i_q current loops
- current, velocity and position loop tuning in practice
- feed-forward, resonance handling and auto-tuning

Bring with you: the delay formula $360° f_c T_d$, the notch filter and the cascade structure — Day 4 tunes exactly these loops.

Figure credits

All other diagrams and plots: own drawings, computed for this course (TikZ / pgfplots / circuitikz).

File	Source (Wikimedia Commons)	Author, licence
d3_aliasing.png	File:AliasingSines.svg	Moxfyre, CC BY-SA 3.0
d3_servo_drives.png	File:Servo motor drive.png	Tero Kontkanen, CC BY-SA 3.0
d3_encoder.jpg	File:Rotary Encoder.jpg	Projuktiponno, CC BY-SA 4.0
d3_servo_drive_motor.jpg	File:Yeskawa Servo Drive & Motor.jpg	Projuktiponno, CC BY-SA 4.0
d3_diode_module.jpg	File:Diode-module.jpg	Hiro36, CC BY-SA 3.0
d3_dclink_caps.jpg	File:Electrolytic capacitors mounted on a PCB.jpg	Giovanna 27, CC BY 4.0
d3_brake_resistor.jpg	File:Turbrake.jpg	Glogger at English Wikipedia, CC BY-SA 3.0
d3_igbt_module.jpg	File:MITSUBISHI IGBT module.jpg	Sthdhk, CC BY-SA 4.0
d3_igbt_module_open.jpg	File:CM600DU-24NFH.jpg	Pslawinski, CC BY-SA 3.0

References

- K. Ogata, *Modern Control Engineering*, 5th ed., Prentice Hall, 2010.
- G. F. Franklin, J. D. Powell, A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 8th ed., Pearson, 2019.
- G. F. Franklin, J. D. Powell, M. Workman, *Digital Control of Dynamic Systems*, 3rd ed., Addison-Wesley, 1998.
- K. J. Åström, R. M. Murray, *Feedback Systems*, 2nd ed., Princeton Univ. Press, 2021.
- K. J. Åström, T. Hägglund, *Advanced PID Control*, ISA, 2006.
- K. J. Åström, B. Wittenmark, *Adaptive Control*, 2nd ed., Addison-Wesley, 1995.
- J. J. E. Slotine, W. Li, *Applied Nonlinear Control*, Prentice Hall, 1991.
- P. A. Ioannou, J. Sun, *Robust Adaptive Control*, Prentice Hall, 1996.
- K. Zhou, J. C. Doyle, K. Glover, *Robust and Optimal Control*, Prentice Hall, 1996.
- B. D. O. Anderson, J. B. Moore, *Optimal Control: Linear Quadratic Methods*, Prentice Hall, 1990.
- D. G. Luenberger, "Observing the state of a linear system," *IEEE Trans. Mil. Electron.*, 8(2), 1964.
- K. Ohnishi, M. Shibata, T. Murakami, "Motion control for advanced mechatronics," *IEEE/ASME Trans. Mechatronics*, 1(1), 1996.
- G. Ellis, *Control System Design Guide*, 4th ed., Butterworth-Heinemann, 2012.
- A. V. Oppenheim, R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed., Pearson, 2010.
- N. Mohan, T. M. Undeland, W. P. Robbins, *Power Electronics: Converters, Applications and Design*, 3rd ed., Wiley, 2003.
- R. Krishnan, *Electric Motor Drives: Modeling, Analysis, and Control*, Prentice Hall, 2001.
- W. Leonhard, *Control of Electrical Drives*, 3rd ed., Springer, 2001.
- L. Balogh, *Fundamentals of MOSFET and IGBT Gate Driver Circuits*, Texas Instruments, SLUA618.
- IEC 61800-5-1 (drive safety), IEC 61800-7 (drive profiles, incl. CiA 402).